AllChrono

Senior / Lead Full-Stack Engineer — Take-Home Project

The Verified Trade Pipeline

A two-surface NestJS + Next.js exercise designed to evaluate domain modelling, transactional integrity, frontend architecture, and engineering taste at the lead level.

1. Context

AllChrono is building the trusted infrastructure for the secondary luxury watch market — a regulated escrow, authentication, and cross-border settlement layer for a category historically run on PDFs, WhatsApp, and seller goodwill. The product surface includes a seller-facing console (list a watch, see consignment status, pull payouts) and a buyer-facing checkout (browse verified inventory, fund escrow, take delivery). Behind both sits a single transactional spine — the **Trade** — that moves through authentication, escrow funding, shipment, and release with strict invariants at every step.

This take-home asks you to build a vertical slice of that spine. You will design the domain model, implement the backend services, expose the API, and ship two real frontend surfaces against it. The blockchain element of AllChrono's product — the per-watch passport that travels with the asset — appears here as a thin, append-only ledger module. You are not being evaluated as a blockchain engineer. You are being evaluated as the engineer who has to make the rest of the system work cleanly around it.

What we are evaluating

- Domain modelling and DDD discipline — bounded contexts, aggregates, value objects, and the discipline to keep business invariants inside the domain rather than scattered across controllers.
- NestJS production patterns — module structure, dependency injection, guards, interceptors, event-driven flows, and transaction handling.
- Next.js application architecture — App Router, RSC vs client boundaries, data-fetching strategy, mutation/cache invalidation, SEO-correctness on public pages, perceived performance on private ones.
- Engineering judgement — what to build well, what to stub honestly, what to leave a decision record about.
- Communication — the README, the ADRs, the commit history, and the code itself should explain the why, not just the what.

What we are not evaluating

- Pixel-perfect design. The UI must be clean and coherent, but no design system is required. Tailwind and shadcn/ui are fine; raw CSS is fine; we care about layout discipline and information hierarchy, not brand polish.
- Production-grade infrastructure. Docker Compose is enough. No Kubernetes, no real cloud, no CI required.
- Real cryptocurrency, real KYC, real payment-gateway integration. All of these are simulated and documented as such.

- Blockchain depth. The passport module exists to show you can integrate a quirky subsystem cleanly. It is not a Web3 exercise.

2. The Domain

Read this section carefully. The names, states, and invariants here are the spec. Deviating without a written reason in your ADR is a signal we read against you.

Core entities

Entity	What it represents	Owns
Watch	A physical luxury watch listed on the platform.	Reference number, brand, model, asking price, condition, listing photos, current Passport reference.
Passport	The append-only history record for a single watch. Survives ownership changes.	Serial fingerprint, ordered list of PassportEntry records, each cryptographically chained to the previous.
Seller / Buyer	Two roles a User can hold in a given Trade. A User may be both across different trades.	Identity, KYC status (simulated), payout / funding method (simulated).
Trade	The transactional aggregate. One Watch, one Seller, one Buyer, one lifecycle.	State, money amounts, timestamps, references to authentication and shipment records, dispute window.
AuthenticationReport	The verifier's verdict on a specific Watch for a specific Trade.	Verdict (PASS / FAIL / INCONCLUSIVE), authenticator id, notes, photo hashes.
LedgerEntry	One row in the append-only passport ledger.	Type (AUTHENTICATED / SERVICED / TRANSFERRED / RE_AUTHENTICATED), payload, prev_hash, this_hash, signer.

The Trade state machine

This is where most candidates lose points. Read every transition and every guard. Do not invent extra states. If you think a state is missing, raise it in your ADR rather than adding it silently.

State	Meaning	Allowed next states
DRAFT	Seller has created the trade against a watch but has not yet submitted it for authentication.	PENDING_AUTH, CANCELLED

PENDING_AUTH	Trade is queued for the authentication step. Buyer is committed but funds are not yet held.	AUTH_PASSED, AUTH_FAILED, CANCELLED
AUTH_PASSED	Authentication verdict is PASS. Buyer must fund escrow within a deadline.	ESCROW_FUNDED, EXPIRED
AUTH_FAILED	Authentication verdict is FAIL or INCONCLUSIVE. Terminal for this trade.	(terminal)
ESCROW_FUNDED	Buyer has funded escrow. Seller must hand the watch to the shipping partner within the SLA.	SHIPPED, REFUNDED_PRE_SHIP
SHIPPED	Watch has been picked up by the shipping partner. Tracking begins.	DELIVERED, LOST_IN_TRANSIT
DELIVERED	Watch has been delivered to the buyer. The dispute window opens.	RELEASED, DISPUTED
DISPUTED	Buyer raised a dispute inside the dispute window. Funds are held.	RELEASED, REFUNDED_POST_DELIVERY
RELEASED	Funds released to seller. Passport ownership transferred. Terminal (happy path).	(terminal)
REFUNDED_PRE_SHIP / REFUNDED_POST_DELIVERY / EXPIRED / CANCELLED / LOST_IN_TRANSIT	Unhappy-path terminals. Each has different bookkeeping consequences.	(terminal)

Hard invariants

These must hold at all times. Violating any of them in your tests or in a manual demo is a hard fail, regardless of how clean the rest of the code is.

1. A Watch can be the subject of at most one non-terminal Trade at a time. Two buyers cannot both be in PENDING_AUTH against the same watch.
2. ESCROW_FUNDED is the only state from which funds can flow to the seller's balance, and only via the RELEASED transition. No code path may credit the seller from any other state.
3. The Passport ledger is append-only. No update, no delete, no reordering. Every entry's prev_hash must equal the SHA-256 of the previous entry's canonical JSON form. A REJECTED transition appends a correction entry; it does not mutate history.

4. Buyer and Seller see different projections of the same Trade. The buyer sees their funded amount and the dispute deadline; the seller sees their net payout (gross minus commission) and the shipment SLA. Neither sees the other's payment instrument details.
5. The escrow-funding endpoint must be idempotent on a buyer-supplied idempotency key. A retried request with the same key and the same body must return the original result, not double-fund. A retried request with the same key and a different body must fail with a 409.
6. Commission is 7% of gross. This is a constant for this exercise; do not over-engineer a pricing engine.

3. What to Build

Backend (NestJS, PostgreSQL)

Modules

- trading — the Trade aggregate, state transitions, commission calculation.
- watches — Watch listings and the link to the current Passport.
- passport — the append-only ledger. Verifies chain integrity on read. Exposes a tiny public lookup endpoint by watch serial that requires no auth (this is the "anyone can read the passport" promise from the product memo).
- authentication — accepts a verdict for a trade. In real life this is a partner integration; here it is a stub admin endpoint that any panel member can hit.
- escrow — funding, holding, releasing, refunding. Idempotency lives here.
- identity — minimal users, JWT-based auth, role guards. A User has one or more Roles (BUYER, SELLER, AUTHENTICATOR, ADMIN).

API surface (minimum)

```
POST   /auth/login                          — issue JWT
POST   /watches                             — seller creates a listing
GET    /watches?status=...                  — buyer-facing catalogue (RSC-friendly)
GET    /watches/:id                         — public watch detail
POST   /trades                              — buyer initiates a trade against a watch
POST   /trades/:id/submit-for-auth          — seller transitions to PENDING_AUTH
POST   /trades/:id/authentication-verdict   — authenticator records PASS/FAIL/INCONCLUSIVE
POST   /trades/:id/fund-escrow              — buyer funds; requires Idempotency-Key header
POST   /trades/:id/mark-shipped             — seller marks shipped (with tracking number)
POST   /trades/:id/mark-delivered           — shipping webhook stub
POST   /trades/:id/release                  — buyer accepts; or auto-release after window
POST   /trades/:id/dispute                  — buyer disputes inside window
GET    /trades/:id                          — projection depends on caller role
GET    /passport/by-serial/:serial          — public; returns the verified chain
```

Frontend (Next.js 14+, App Router)

Two surfaces, both required. They share components where it makes sense and diverge where the product diverges.

Surface A — Seller Console (authenticated)

- Dashboard listing the seller's watches and their active trades, with state-aware actions on each row (a SHIPPED trade shows a tracking link; an ESCROW_FUNDED trade shows a "Mark shipped" CTA with a tracking-number form; an AUTH_FAILED trade shows the reason).

- Create-listing flow. Multi-step form with client-side validation. Optimistic UI on submit, with a rollback path on server error.
- Trade detail view showing the seller's projection — net payout, commission line item, shipment SLA countdown, passport entry history for that watch.

Surface B — Buyer Checkout (authenticated for purchase, public for browse)

- Public catalogue page rendered as a Server Component. Must be SEO-correct: meaningful title, description, canonical, OpenGraph, structured data for each listing. Pagination via search params, not client state.
- Public watch detail page. Server-rendered. Must include the passport history (read from the public passport endpoint). This page is the buyer's primary trust signal — treat it accordingly.
- Authenticated checkout flow: initiate trade, wait for authentication verdict, fund escrow. The fund-escrow action must implement the idempotency-key contract on the client side too — a double-click or a flaky network must not double-fund.
- Post-purchase view: dispute-window countdown, "release funds" CTA, "raise dispute" CTA with a reason.

Blockchain element (kept deliberately small)

The Passport module is the only place this shows up. It is not on a real chain. You implement the chain as an append-only Postgres table with a hash chain (each row's prev_hash equals the SHA-256 of the previous row's canonical JSON form, signed with a static keypair loaded from environment). The /passport/by-serial/:serial endpoint must verify the entire chain on read and return a verified=true|false flag. A tampered row must cause verified=false without throwing.

That is the entire blockchain scope. Do not pull in ethers.js, do not deploy to a testnet, do not write Solidity. If you have Web3 experience and want to flag it, do so in your ADR — that is a hiring-signal note, not a code change.

4. Deliverables

7. A single Git repository (GitHub, GitLab, or a zip with .git included). Monorepo or two folders — your call. Document the choice in the README.
8. docker-compose.yml that brings up Postgres and any other service your app needs. README must include the single command that takes a reviewer from clone to running app.
9. Seed script that creates: one seller, one buyer, one authenticator, three watches in different states (one DRAFT, one with an active SHIPPED trade, one with a RELEASED trade and a populated passport). Reviewers should be able to demo the happy path and the unhappy paths without manually clicking through setup.
10. README covering: how to run, how to test, the three most important design decisions you made and why, and the three things you would do next with more time.
11. At least two ADRs (Architecture Decision Records) in docs/adr/. Each one paragraph. Examples: "Why TypeORM over Prisma here", "Why event-driven state transitions", "Why we kept the passport hash chain in Postgres rather than Redis".
12. A short LOOM or written walkthrough (max five minutes / one page) of: the state machine, the idempotency design, and one thing you are not proud of and would change.

Time expectation

We expect this to take **two to four focused hours** for an engineer at the level we are hiring for. If it is taking you longer, stop and submit what you have with a note explaining what you would do next. We would rather see a smaller, sharper deliverable with honest gaps than a sprawling one with hidden ones. AI-assisted engineering is welcome and expected; we will ask you in the live round to walk us through any code you did not personally write.

5. Ground Rules

AI tools

Use them. Cursor, Claude, Copilot, whatever you prefer. We use them every day. The live round will test whether you understand the code in your own repository — not whether you wrote every keystroke. If you cannot explain a transaction boundary in your own escrow service, it does not matter who wrote it.

Stack

- Backend: NestJS, PostgreSQL. ORM is your choice — TypeORM, Prisma, MikroORM, or raw SQL with a query builder are all acceptable. Justify the choice in an ADR.
- Frontend: Next.js 14+ (App Router required), React 18+, TypeScript. UI library is your choice.
- Testing: Jest or Vitest for unit; Supertest or Playwright for integration; pick one and run with it.
- Auth: a real JWT implementation with role guards. Refresh tokens optional.

Submission

- Reply to the recruiting thread with the repository URL and a one-paragraph note: anything we should know before opening it, anything you cut for time, anything you are particularly proud of.
- Do not push directly before the panel session; the panel will pull from the SHA you submit.

Plagiarism and templates

Starter templates are fine if you note them. Wholesale copying of a public NestJS marketplace example is not — and they all have the same fingerprints, so we will recognise them. If you have built something similar in the past, say so and tell us what you reused.

6. A Note From the Hiring Team

This project is deliberately under-specified in a few places. We have told you what the domain is, what the invariants are, and what the surfaces must do. We have not told you how to lay out the modules, what your event bus should look like, where to draw the line between a domain event and a public API event, or how to make the passport endpoint feel fast when the chain gets long.

Those are the decisions a lead engineer makes. Make them, write them down, and be ready to defend them. We are not looking for the one right answer — there isn't one. We are looking for the engineer who can explain why their answer is the right one for the constraints we have given.

Good luck. We are looking forward to reading your code.